

UNFILTERED - Final Project Report

Boston University | CS 501 | Spring 2026

By - Vidney Jadhav , Ashish Joshi and Kaushik Joshi

1. Problem Definition and Motivation

Gen-Z is the most digitally connected generation, but it also reports high levels of loneliness, anxiety, burnout, and emotional isolation. Many students spend hours online while still feeling unheard or emotionally disconnected. Existing platforms often intensify this problem: social media can create unhealthy comparison loops, open forums may feel unsafe or unmoderated, and professional therapy platforms can be costly or intimidating for students who need lightweight, everyday support.

Unfiltered was designed to close this gap by providing an anonymous, youth-first, emotionally safe mobile environment where users can track moods, express thoughts through journaling, connect with peers in moderated topic rooms, and receive gentle nudges toward healthy offline behavior.

Key design goals of the app are:

- Encourage honest emotional expression without social pressure.
- Reduce comparison and performative behavior common in public social feeds.
- Support self-awareness through mood and journaling patterns.
- Promote healthier digital habits through location-based detox suggestions.

2. Architecture

The project follows a modular MVVM (Model-View-ViewModel) architecture with a repository layer, designed for scalability, clearer testing boundaries, and maintainable feature growth.

View Layer (Jetpack Compose)

The UI is implemented fully in Jetpack Compose, covering authentication, mood logging, journaling, chat, and maps. Composable screens observe state and render deterministic UI updates based on current screen state.

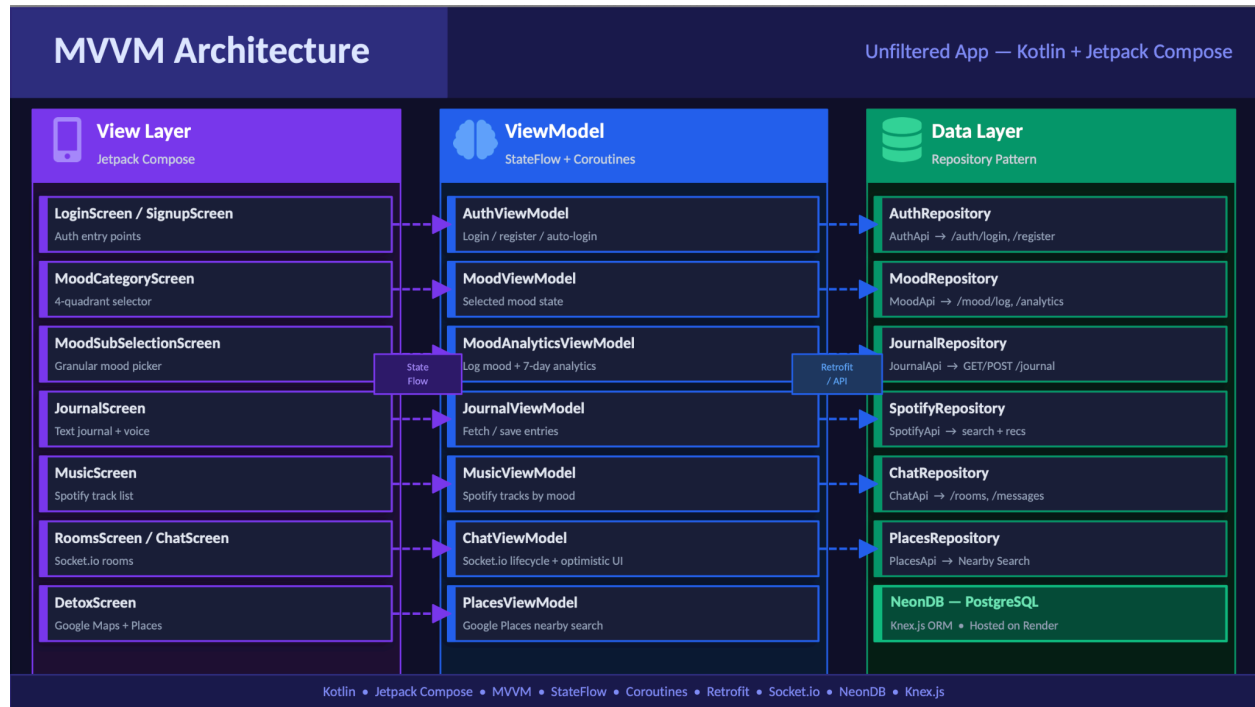
ViewModel Layer (StateFlow + Coroutines)

ViewModels orchestrate UI state and feature-specific business logic. StateFlow is used for reactive state emission, while Kotlin coroutines manage asynchronous network calls and socket events. This reduces UI-thread blocking and keeps screen behavior consistent under loading, success, and error states.

Data Layer (Repository Pattern)

Repositories abstract external interactions, including REST APIs (Retrofit), real-time messaging (Socket.IO), and third-party integrations (Spotify and Google Places). This indirection decouples UI logic from transport details and simplifies unit and integration testing.

Overall, the architecture improves modularity, feature ownership, and long-term maintainability by preserving strict separation of concerns.



3. APIs and Sensor Usage

Authentication APIs (`/api/auth`)

Method	Endpoint	Description
POST	/api/auth/register	Registers a new user account (public endpoint).
POST	/api/auth/login	Authenticates credentials and returns JWT token (public endpoint).
GET	/api/auth/me	Returns authenticated user profile (JWT required).

Mood APIs (`/api/mood`)

Method	Endpoint	Description
POST	/api/mood/log	Stores mood type/sub-type entry for user analytics (JWT required).

GET	/api/mood/analytics	Returns aggregated mood trends; supports optional days filter (JWT required).
-----	---------------------	---

Journaling APIs (`/api/journals`, `/api/journal`)

Method	Endpoint	Description
POST	/api/journals	Creates a journal entry for the logged-in user.
GET	/api/journals	Fetches all journals for the logged-in user.
POST	/api/journal/entries	Creates a normalized journal entry record.
GET	/api/journal/entries	Fetches normalized journal entry history.

Chat APIs and Real-Time Events

Method	Endpoint	Description
GET	/api/rooms	Returns available chat rooms for emotional topics.
GET	/api/rooms/{id}/messages	Returns message history for a selected room.

Socket Event	Description
join_room	Client joins a room channel using room ID to receive live updates.
send_message	Client sends message payload; server persists and broadcasts in real time.

Sensor Usage

- **GPS / Location** (ACCESS_FINE_LOCATION and ACCESS_COARSE_LOCATION): powers Digital Detox recommendations by detecting nearby parks, cafes, libraries, and calm spaces via map-based discovery.
- **Microphone**: enables voice journaling through speech-to-text, lowering friction for users who can express emotions more naturally by speaking than typing.

4. Database Usage and Schema

The backend uses PostgreSQL hosted on Neon as the system of record. The Android client does not maintain a heavy local relational database; instead it relies on API-first synchronization for consistency across devices and sessions.

Data access occurs through Express.js endpoints for CRUD and analytics operations, while Socket.IO handles low-latency bidirectional chat. This hybrid communication model balances reliable persistence with responsive interactions.

Core relational schema overview:

Table	Purpose	Important Columns / Constraints
users	Authentication and profile identity	id (PK), email UNIQUE NOT NULL, password_hash NOT NULL, name, created_at
journals	User journal content with mood label	id (PK), user_id (FK users), content, entry_type, mood_label, created_at
journal_entries	Normalized journal entry records	id (PK), user_id (FK users) NOT NULL, content NOT NULL, created_at
daily_moods	Numeric mood snapshots for trends	id (PK), user_id (FK users), mood_score NOT NULL, created_at
mood_logs	Detailed categorical mood logging	id (PK), user_id (FK users), mode_type NOT NULL, mode_sub_type NOT NULL, created_at
rooms	Chat room definitions by theme	id (PK), name NOT NULL, mood_tag NOT NULL, created_at
messages	Room message persistence	id (PK), room_id (FK rooms) NOT NULL, user_id (FK users) NOT NULL, content NOT NULL, created_at

Foreign-key constraints and cascade rules were chosen intentionally to preserve referential integrity while simplifying cleanup behavior (for example, message deletion when a room is removed).

5. Team Responsibilities and Contributions

Development followed a role-oriented model with shared integration checkpoints, where each member owned a major subsystem and coordinated through interface contracts.

Kaushik Sandeep Joshi - Backend and APIs

- Built the Express.js backend with login, register, and user profile endpoints.
- Designed and migrated PostgreSQL schema (Knex + NeonDB).
- Implemented route modules for auth, moods, journals, and chat rooms.

- Deployed backend services on Render and validated APIs with Postman.

Ashish Sandeep Joshi - UI/UX and Frontend

- Developed 10+ Jetpack Compose screens with Material 3 components.
- Designed the mood quadrant interaction model and screen theming consistency.
- Implemented key experience surfaces including room listings and media components.
- Focused on readability, accessibility, and user-friendly flow transitions.

Vidney Milind Jadhav - Architecture and State Management

- Implemented NavHost graph and typed route-based navigation flow.
- Built feature ViewModels with StateFlow and coroutine-driven state updates.
- Integrated Socket.IO real-time chat flow in the app state lifecycle.
- Integrated Spotify credential flow and coordinated multi-module wiring.

6. AI Reflection

How AI was used across the semester

- Used as a guided tutor for Android architecture patterns (MVVM, StateFlow, coroutines).
- Used for ideation: evaluating feature options for mood tracking, journaling, and chat.
- Used for debugging support in Kotlin, Gradle configuration, and API integration errors.
- Used for implementation scaffolding, then manually adapted to project constraints.

Where AI influenced architecture, code, testing, and UX

- Architecture: reinforced repository boundaries and ViewModel ownership.
- Code: accelerated boilerplate generation and error-handling patterns.
- Testing: helped define API verification flows and debugging checklists.
- UX: suggested refinements in form flow, state feedback, and screen consistency.

What AI helped accelerate

- Faster onboarding to unfamiliar libraries and integration patterns.
- Reduced turnaround time from bug discovery to working fix.
- Improved iteration speed for both backend and frontend milestones.

Why certain AI suggestions were rejected

- Some outputs were generic and not aligned with this app's architecture.
- Some suggestions introduced patterns that conflicted with existing module design.
- Some recommendations did not satisfy assignment constraints or project scope.
- Final decisions prioritized correctness, maintainability, and team understanding over speed.

7. Conclusion

Unfiltered demonstrates a complete mobile system that combines emotional well-being goals with production-style Android engineering practices: modern architecture, secure authentication, cloud-backed relational data, real-time communication, and reflective human-centered design. The final implementation also shows how responsible AI assistance can accelerate development when combined with deliberate engineering judgment.